Infrastructure as Code: Foundations

From cloud strategy to automated delivery

SEAtS | 749131

University of Southampton

Intended Learning Outcomes

By the end of this lecture you will be able to:

1. Summarise cloud characteristics and deployment/service models that influence IaC.
2. Explain how DevOps and "cattle not pets" thinking enable reproducible infrastructure.
3. Decompose infrastructure requirements into declarative modules and interfaces.
4. Compare manual, ad hoc, and Infrastructure-as-Code approaches, and the three ways to *write* IaC.
5. Trace the Pulumi project/stack workflow from `pulumi new` through `up` to `destroy`.
6. Plan testing, CI/CD, and policy-as-code workflows for IaC.
7. Identify common risks/anti-patterns in IaC projects and propose mitigations.

© University of Southampton

Agenda

1. Cloud context: characteristics, deployment/service models.
2. DevOps mindset & naming the IaC problem.
3. IaC building blocks, modules, interfaces.
4. Pulumi fundamentals: declarative vs imperative, projects and stacks.
6. Testing, CI/CD, and policy-as-code.
7. Risks, anti-patterns, and closure.

© University of Southampton

What is Cloud Computing?

"A model for enabling convenient, on-demand network access to a shared pool of configurable computing resources… that can be rapidly provisioned and released with minimal management effort or service provider interaction." — NIST SP 800-145.

1. On-demand self-service
2. Broad network access
3. Resource pooling
4. Rapid elasticity
5. Measured service

- Improved security and "unlimited" capacity.
- Cost efficiencies; zero up-front capital.
- Elastic scaling as system grows.
- Time efficiencies via virtual resources.
- Power efficiency and visibility into consumption.
- Improved process control and quick recovery.

© University of Southampton

timeline title Evolution of Cloud Infrastructure Wave 1 : Static infrastructure : Single VMs accessed via SSH Wave 2 : Growing complexity : Containers : Provisioning tools (Docker, Datadog) Wave 3 : Modern cloud : Containers : Serverless : Managed services

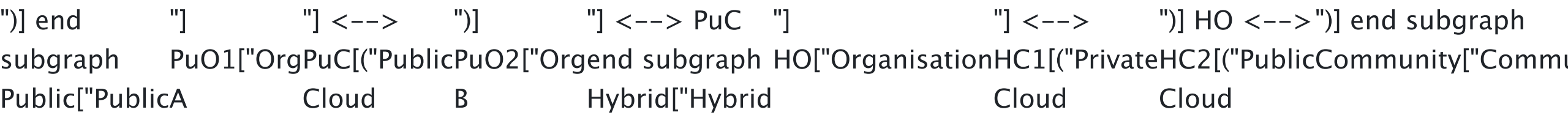
© University of Southampton

Each wave adds more moving pieces — and more edges connecting them. **IaC is the glue between those edges.** Counter-intuitively, a more complex-looking modern architecture diagram can be *easier* to maintain than an old simple one, because the slice of it that's actually your own hand-written code (as opposed to managed service configuration) keeps shrinking — less operational burden sits on your team as the cloud provider absorbs more of the plumbing.

© University of Southampton

Cloud Deployment Models

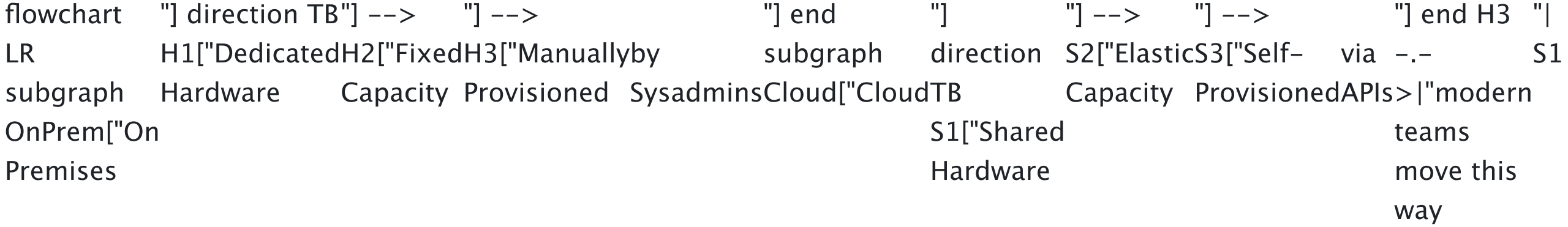
1. **Public:** Infrastructure available to general public, owned by service provider.
2. **Private:** Dedicated to a single organisation, managed on/off premise.
3. **Community:** Shared by organisations with common concerns.
4. **Hybrid:** Combination of cloud types.



© University of Southampton

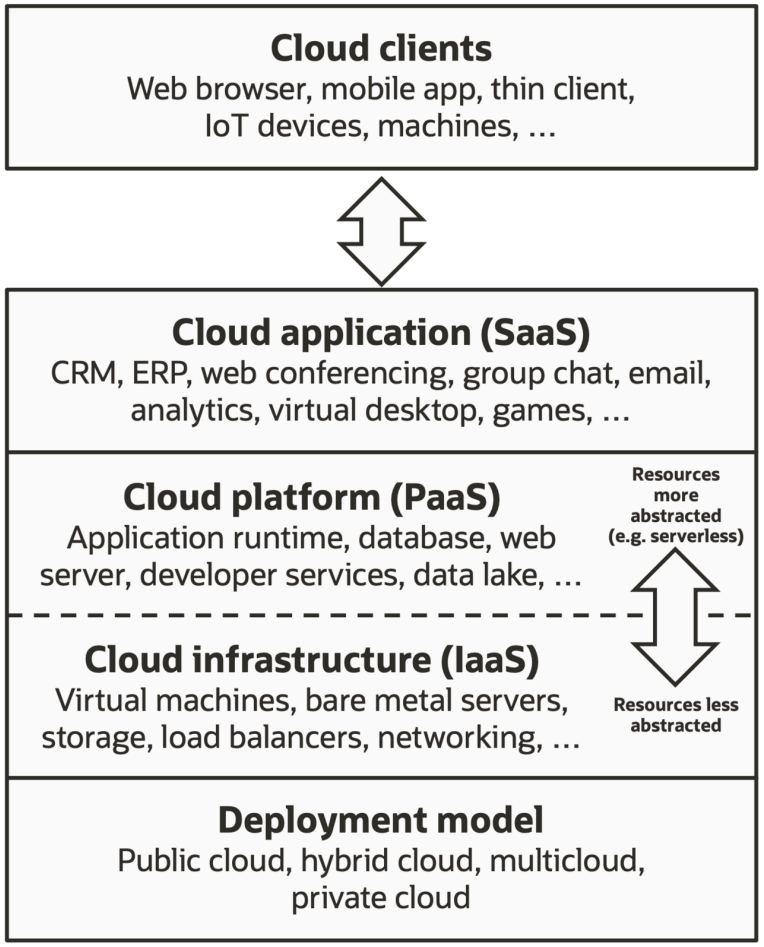
On premises Computing vs. Cloud Computing

On Premises	Cloud
Manually provisioned	Self-provisioned
Dedicated hardware	Shared hardware
Fixed capacity	Elastic capacity
Pay for capacity	Pay for use
CapEx + OpEx	Operational expenses
Managed via sysadmins	Managed via APIs



© University of Southampton
University of Southampton

Main Service Delivery Models



IaaS: Provision virtual machines, networks, and storage inside the provider's infrastructure to run any OS or application stack.

- Control over guest OS and runtimes.
- Examples: EC2, Virtual Networks.

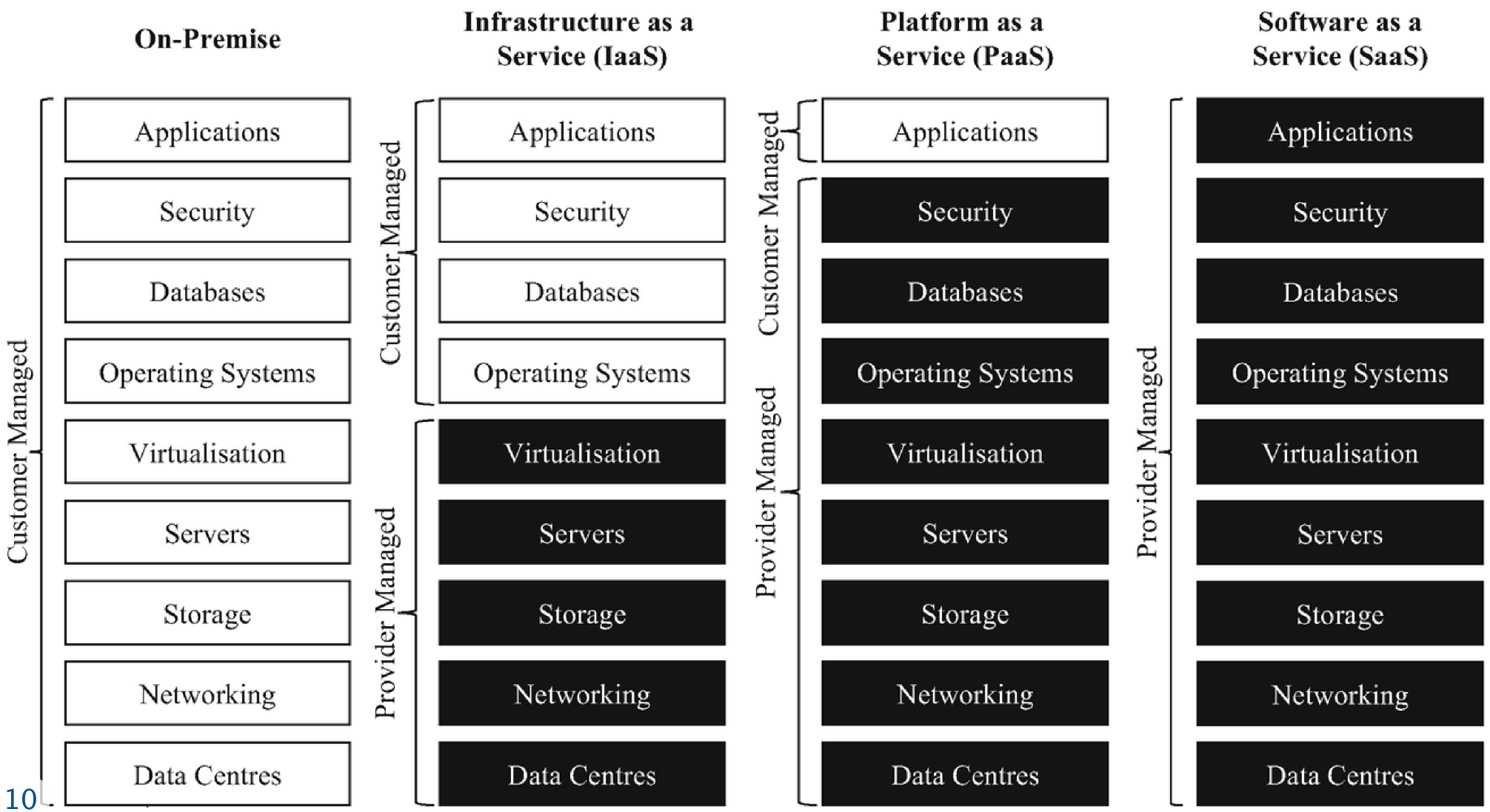
PaaS: Build and deploy custom applications using provider-supported runtimes and tooling.

- Auto elasticity and built-in CI/CD.
- Examples: App Engine, Azure App Service.

SaaS: Consume fully managed provider applications delivered over the cloud.

- No infrastructure management; pay per use.
- Examples: Gmail, Salesforce.

© University of Southampton



© University of Southampton
University of Southampton

Service Delivery Models — Who Manages What

IaaS: You manage from the OS up — runtime, middleware, data, and the application.

PaaS: You manage your application and data; the provider manages the OS, runtime, and middleware.

SaaS: The provider manages everything — you just use the application.

IaC's job: write the code that manages the **IaaS resources** — and the SaaS-like managed services — a cloud provider gives you. The inner the layer, the more of it your Pulumi/Terraform code is responsible for.

© University of Southampton

Architectural Patterns

Pattern	Description
Peer-to-Peer	Direct communication between two components without a central coordinator.
API Gateway	Single entry point for all client requests to the backend services of an application.
Pub-Sub	Decouples publishers from subscribers through a message broker.
Request-Response	Client sends a request to a server and waits for a response.
Event Sourcing	Store application state changes as a sequence of immutable events.
ETL	Gather data from sources, transform it, and load into a destination.
Batching	Accumulate data over time/threshold before processing as a group.
Streaming Processing	Continuously ingest, process, and analyse data in real time.
Orchestration	Central coordinator manages interactions between distributed components.

© University of Southampton

Architectural Patterns — Service Communication (1/3)

Peer-to-Peer

```
flowchart LR
O1["Order Svc 1"] -.-> P1["Payment Svc 1"]
O1 -.-> P2["Payment Svc 2"]
O2["Order Svc 2"] -.-> P2
O2 -.-> P3["Payment Svc 3"]
O3["Order Svc 3"] -.-> P3
O3 -.-> P4["Payment Svc 4"]
```


© University of Southampton

API Gateway

```
flowchart LR Client --> "] GW --> "] GW --> "] GW --> "] GW -.-> Cross["Auth · "]
GW["API Gateway S1["Service 1 S2["Service 2 S3["Service 3 Rate Limit · Routing
```


© University of Southampton

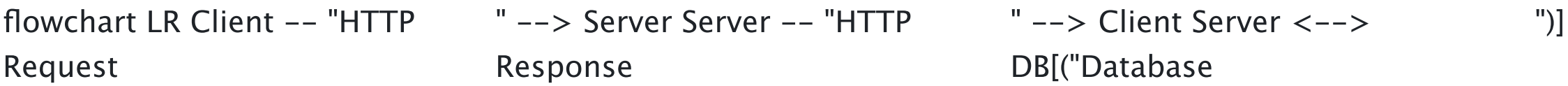
Pub-Sub

```
flowchart LR
Pub1["Publisher 1
"] --> Topic(("Topic
")) Pub2["Publisher 2
"] --> Topic Topic --> Sub1["Subscriber 1
"] Topic --> Sub2["Subscriber 2
"]
```


© University of Southampton

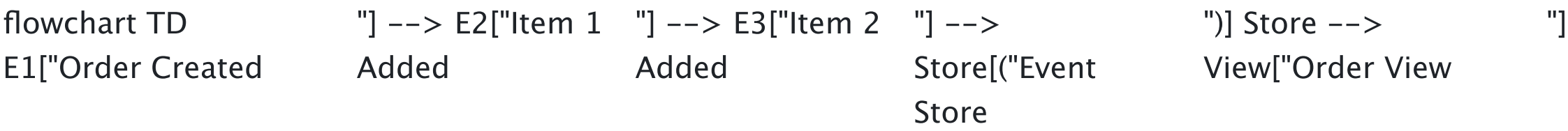
Architectural Patterns — Data Flow (2/3)

Request–Response



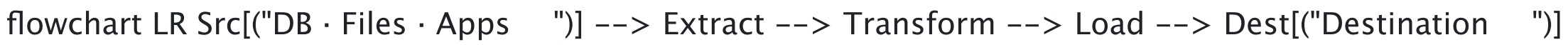
© University of Southampton

Event Sourcing



© University of Southampton

ETL



© University of Southampton

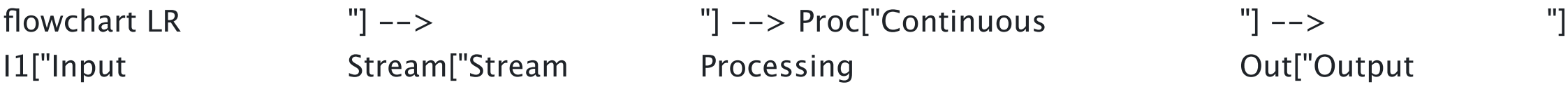
Architectural Patterns — Processing & Control (3/3)

Batching

flowchart LR I1["Input "] & I2["Input "] & I3["Input "] --> BP["Batch Processor "] --> Out["Output "]

© University of Southampton

Streaming Processing



© University of Southampton

Orchestration

```
flowchart TD
Orch(("Orchestrator
")) -->
A["Service A
"] Orch -->
B["Service B
"] Orch -->
C["Service C
"] Orch -->
D["Service D
"]
```


© University of Southampton

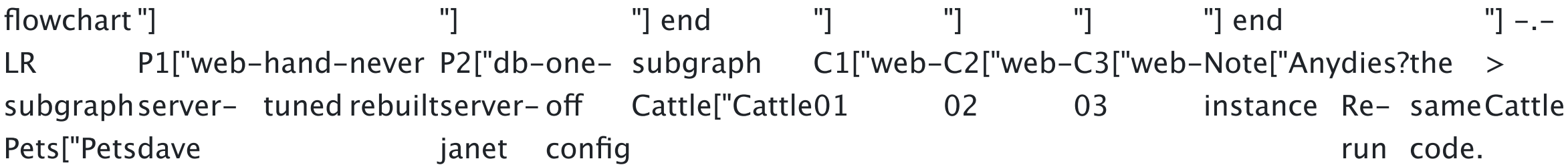
Many "as-a-Service" Offerings

Examples include Analytics, API, AI, Backend, Banking, Big Data, Blockchain, Business Process, Code, Content, Construction, Container, Crane, Edge Compute, Contact Centre, Communications Platform, Data, Desktop, Drone, Database, Distribution, Energy, Engagement, EV, Function, Farming, Firewall, Games, Governance, Hadoop, Hardware, Housing, Identity, Image, IoT, IT, Knowledge, Load Balancing, Logging, Management, Microgrid, Mobility, Monitoring, Metal, Mobile Backend, Machine Learning, Network, Network Defense, Payments, Policy, Product, Push Notification, QA, RAN, Recovery, Recycling, Robot, Search, Security, Solar, Shared Storage, Transportation, Testing, Time Series DB, Unified Communications, and "Anything as a Service."

© University of Southampton

Infrastructure: Cattle vs Pets

Development environments should be consistent and reproducible, like managing "cattle" (identical, disposable instances) rather than unique "pets" (customised desktops). This ensures faster onboarding, predictable testing, and reliable automation.

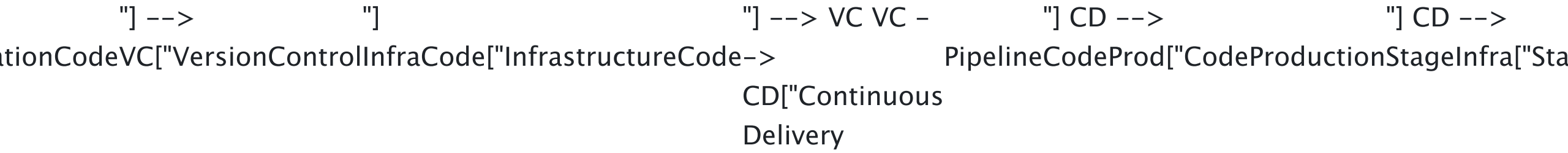


© University of Southampton

DevOps =

Development + Operations

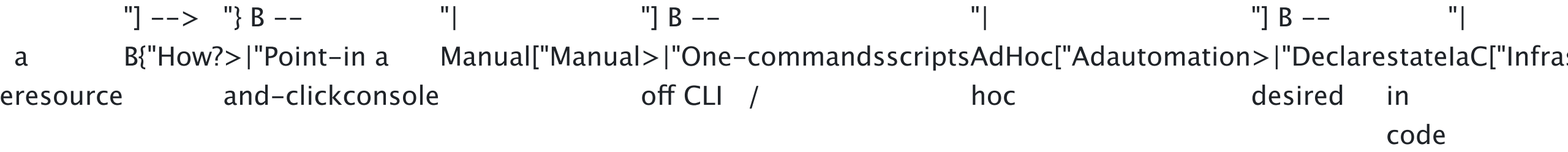
Development: fast, automated builds & tests. **Operations:** reliability, monitoring, incident response. **Bridge:** shared CI/CD + IaC tooling and SLOs align both.



© University of Southampton

Naming the Problem IaC Solves

Modern applications can deploy to production up to **100 times a day**, and infrastructure often needs to scale up or down automatically in response to load — neither is viable with manual, one-off changes.



© University of Southampton

Three Ways to Manage Infrastructure

Approach	What it is
Manual	Point-and-click resource creation/modification directly in a console
Ad hoc automation	CLI commands or scripts — some code, but not structured for reuse
Infrastructure as Code	**Provisioning** (declaratively creating/modifying resources) + **configuration** (changing an existing resource's state)

© University of Southampton

Three Ways toWrite IaC

Method	Description
JSON	Resources (EC2 instance, security group, …) listed as structured data for the provider to consume
Domain-specific language (DSL)	A custom syntax built for one tool/provider — looks code-like, isn't a general-purpose language
General-purpose programming language	A real language (Pulumi supports TypeScript, JavaScript, Python, Go, .NET) used to describe infrastructure

The general-purpose-language approach brings **loops, conditionals, classes, and packages** to infrastructure definitions — e.g. a `for` loop that creates one instance per availability zone — plus normal IDEs, linters, and test frameworks.

© University of Southampton

IaC Definition & Motivation

Infrastructure as Code (IaC): Define and manage infrastructure via machine-readable files instead of manual provisioning.

Why essential?

- Rapid provisioning for software, hardware, and learning integration.
- Reduces configuration drift and enables CI/CD pipelines.
- Enhances repeatability, testing, and formal specification.
- Critical for distributed cloud systems.

© University of Southampton

Requirements in IaC

Functional examples: Deploy VPC/VNet; automate Kubernetes clusters; create IAM roles.

Non-functional examples: Scalability, security hardening, fault tolerance, cost-efficiency, auditability, compliance.

IaC formalises both sets of requirements in code.

- Cloud provider selection (AWS, Azure, GCP).
- State storage (S3, Blob, GCS).
- Secrets management (Vault, Secrets Manager).
- Access patterns (SSH, API gateways, private endpoints).
- Compliance constraints (residency, logging).
- Integration requirements (CI, monitoring, networks).

Methods: stakeholder interviews, architecture diagrams, threat modelling.

© University of Southampton

System Architecture for IaC

IaC encourages decomposition into modules:

- Networking (VPC, subnets, firewalls).
- Compute (VMs, containers).
- Storage (S3, RDS).
- Identity (IAM roles, policies).
- Monitoring (metrics, logging).

Declarative vs Imperative

Approach	Description	Examples
Declarative	Describe desired state; engine reconciles	Terraform, CloudFormation, Bicep
Imperative	Execute procedural steps	Ansible, scripts

© University of Southampton

Designing IaC Systems

Decomposing into Blocks

Example module folders:

- `/networking`
- `/identity`
- `/compute`
- `/database`
- `/monitoring`

Identifying Interfaces

Interfaces via:

- Input variables.
- Output variables.
- Remote state references.

Interfaces must be clear, versioned, documented, and stable.

Designing APIs & Protocols

IaC interacts with:

- Cloud provider APIs.
- Terraform / Pulumi provider schemas.
- Kubernetes manifests/CRDs.

Integrating Providers & IP

- Cloud providers (AWS, Azure, GCP).
- Third-party providers (Cloudflare, Datadog).
- Internal libraries reused across teams.

Managing Third-Party IP

- Version pinning and lifecycle management.
- Licensing and compliance considerations.
- Security scanning of modules/providers.

© University of Southampton

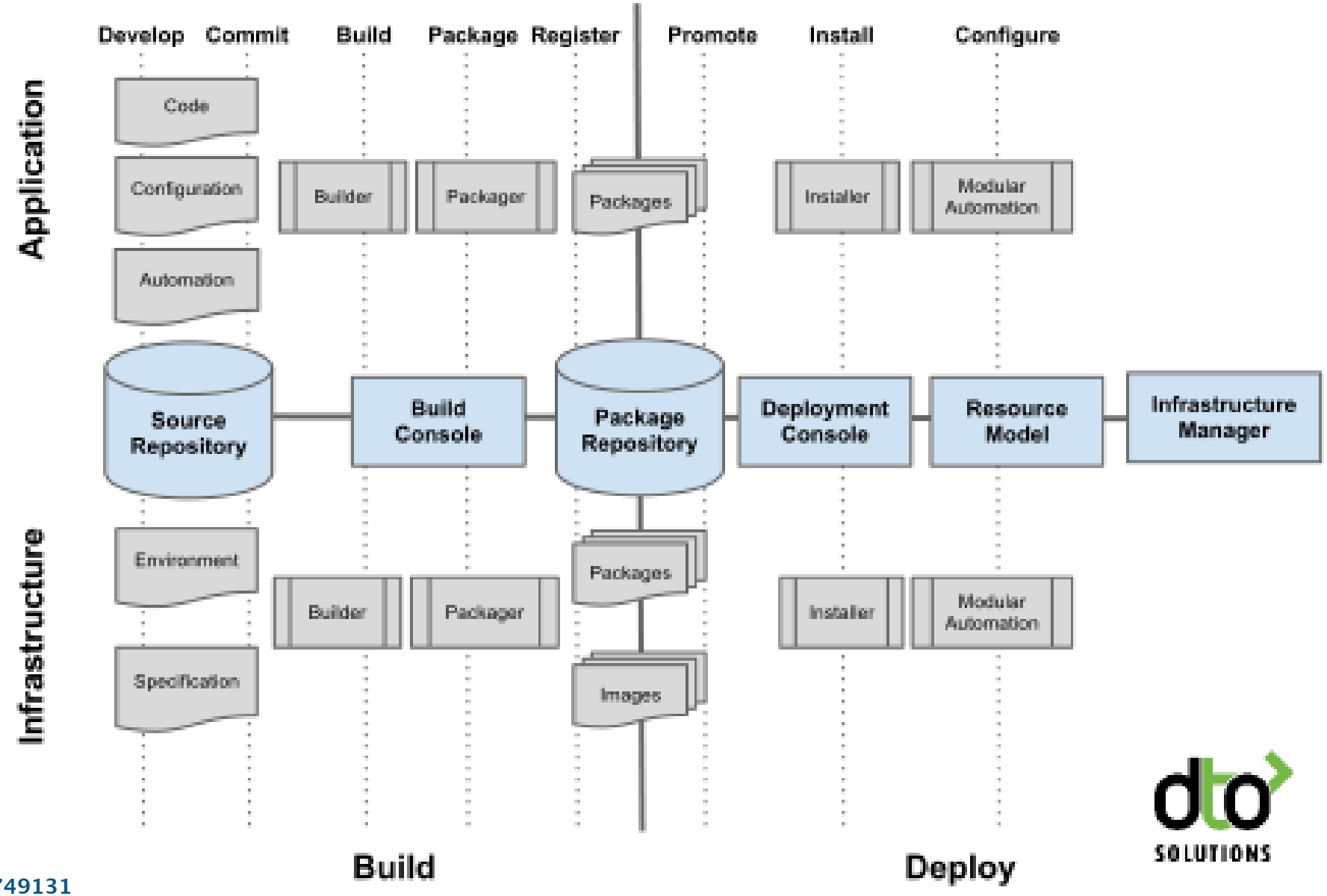
Service Delivery Platform



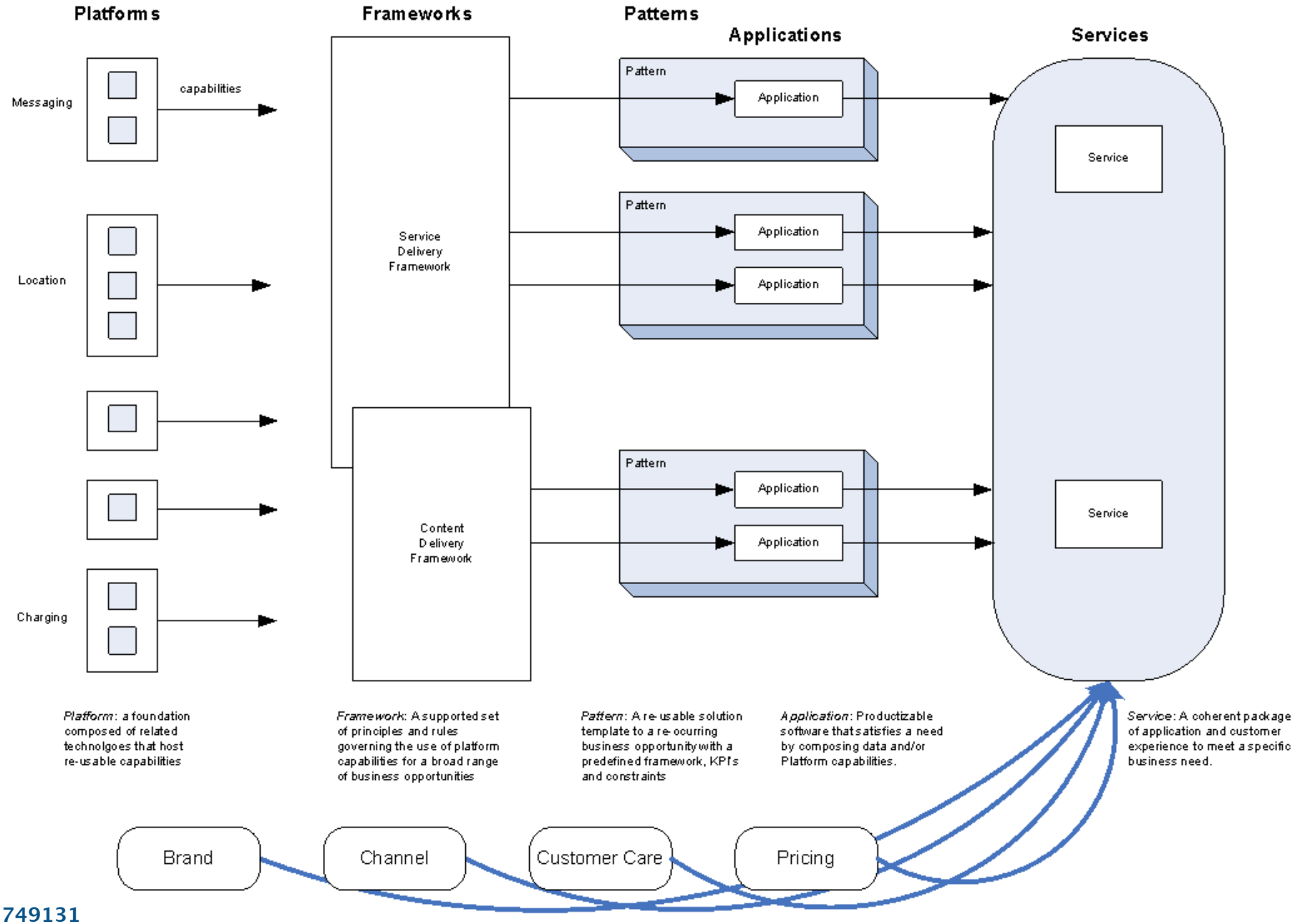
- **Artifact:** Versioned, tested, deployed units.
- **Procedures (Imperative):** Commands to produce desired state (e.g., Bash).
- **Functions (Declarative):** Define desired state; tooling conforms system (e.g., SQL, makefiles).

© University of Southampton

Service Delivery Platform



© University of Southampton



© University of Southampton

Formal Specification in IaC

- JSON/YAML schemas (CloudFormation, Kubernetes).
- Terraform variable typing and module interfaces.
- Policy-as-Code (OPA/Rego) for compliance.
- Cloud policies (AWS Config, Azure Policy) to enforce encryption, logging, segmentation.

Formalisation supports reproducibility and enforcement.

© University of Southampton

IaC as JSON / DSL — Vagrant Examples

```
Vagrant.configure("2") do |config|
  config.vm.box = "peru/ubuntu-18.04-desktop-amd64"
  config.vm.box_version = "2.3"
  config.vm.provider "virtualbox" do |vb|
    vb.gui = true
    vb.memory = "1024"
    vb.name = "ubuntudesktopscyther"
  end
end
```

```
servers = YAML.load_file('servers.yaml')
Vagrant.configure(VAGRANTFILE_API_VERSION) do |config|
  servers.each do |server|
    config.vm.define server["name"] do |srv|
      srv.vm.box = server["box"]
      srv.vm.network "private_network", ip: server["ip"]
      srv.vm.provider :virtualbox do |vb|
        vb.name = server["name"]
        vb.memory = server["ram"]
      end
    end
  end
end
```


© University of Southampton

Pulumi : hybrid?

Pulumi is a hybrid, not a straightforward "imperative" tool: its *state model* is declarative — you describe desired infrastructure and `pulumi up` diffs it against reality, exactly like Terraform — but its *authoring language* is a real general-purpose language (Python), which is imperative. Two separate axes: **what gets reconciled** vs **how you express it**.

Declarative state reconciliation scales better than hand-written procedural steps for large multi-component systems — true whether you write that declaration in HCL (Terraform) or in Python (Pulumi).

© University of Southampton

From Theory to Pulumi: The Workflow

A **project** is a single program that declares desired infrastructure; a **stack** is one deployed instance of that project (e.g. `dev` vs `prod`) — same program, different config (region, parameters), deployed independently. Neither term appears in the theory chapters above — this is the vocabulary that was missing.

sequenceDiagram actor Dev participant CLI as Pulumi CLI participant Cloud as AWS Dev->>CLI: pulumi new <template> CLI-->>Dev: scaffolds project + "dev" stack Dev->>CLI: pulumi config set aws:region ... Dev->>CLI: pulumi up CLI->>CLI: diff desired state (code) vs actual state CLI-->>Dev: preview of resources to create/update Dev->>CLI: confirm (yes) CLI->>Cloud: create/update resources Cloud-->>CLI: resulting resource state CLI-->>Dev: outputs (e.g. bucket name, URL) Dev->>CLI: pulumi stack output <name> CLI-->>Dev: exported value Note over Dev,CLI: edit main.py, re-run "pulumi up" — only the minimal diff is applied Dev->>CLI: pulumi destroy CLI->>Cloud: tear down all stack resources Dev->>CLI: pulumi stack rm <name>

© University of Southampton

Testing IaC Systems

Testing types:

1. Static testing (linting: tflint, checkov; policy-as-code).
2. Unit tests (Terratest, Molecule).
3. Integration tests (ephemeral deployments).
4. Acceptance tests (VPC routing, IAM permissions).
5. Security tests (CSPM, IaC scanning).

© University of Southampton

Build Systems & CI

Standard Pipeline

1. Validate (`pulumi preview` / `terraform validate`).
2. Lint (tflint, checkov).
3. Plan (static analysis, diffs).
4. Approvals (human-in-loop).
5. Apply (automated, controlled).
6. Post-deploy tests (Terratest, smoke tests).

Continuous Deployment (GitOps)

- ArgoCD / Flux for declarative sync.
- Atlantis, Spacelift, Terraform Cloud / Pulumi Cloud for collaboration.
- Automated rollbacks based on policy drivers.

Managing Development

Methodologies: Agile, DevOps, GitOps, infrastructure release management.

- Multi-team decomposition, module backlogs, environment versioning.
- Branch protection, review bots (tfsec), semantic versioning.
- Automated changelogs and release notes.

© University of Southampton

Provisioning / Deployment / Orchestration / CM

"] end "] "] "]
sioningsubgraph Orchestration["Infrastructure:OrchestrationDeployment["Application:DeploymentConfigMgmt["Applica
Deploy["Deploy

- **Provisioning:** Prepare server (hardware, OS, services, network).
- **Deployment:** Automated app deployment & upgrades.
- **Orchestration:** Coordinated operations across systems.
- **Configuration Management:** Maintain app dependencies.

© University of Southampton

Risks & Anti-Patterns

- Configuration drift.
- Hard-coded secrets.
- Unversioned remote state.
- No testing or code reviews.
- Monolithic Terraform/Pulumi files, parallel changes without locking.
- Black-box modules with no documentation.

Mitigation: code reviews, state locking, policy-as-code, decomposition.

© University of Southampton

Summary

- Cloud models set expectations; IaC encodes them into reproducible infrastructure.
- DevOps habits (IaC, automation, observability) keep environments consistent.
- Break infrastructure into declarative modules with typed interfaces and remote state.
- Pulumi's *state model* is declarative even though its *authoring language* is imperative Python — don't conflate the two axes.
- `pulumi new` → `up` → `destroy` is the same provisioning loop regardless of what infrastructure you're standing up.
- Testing, CI/CD, and policy-as-code make IaC trustworthy and auditable.
- Mitigate anti-patterns (drift, secrets, monoliths, unlocked state) through reviews, locking, decomposition, and documentation.

© University of Southampton